


# โครงงาน การเขียนโปรแกรมเชิงอ็อบเจกต์และโครงสร้างข้อมูล

# ระบบจัดการร้านอาหาร ( Restaurant System )

**จัดทำโดย**

1. B6701611 นายณรงค์ชัย ท่าทอง
2. B6702076 นางสาวณัฐกมล เพชร์รัตน์
3. B6703257 นายธณกร ศุภธรรมกิจ
4. B6703738 นายเจษฎากร ทองสืบสาย

**เสนอ**

ผศ. ดร.ศุภกฤษฎิ์ นิวัฒนากูล

ผศ. ดร.จิติมนต์ อั่งสกุล

อ. ดร.ธรรมศักดิ์ เธียรนิเวศน์

รายงานนี้เป็นส่วนหนึ่งของรายวิชา 1101093

โครงงานการเขียนโปรแกรมเชิงอ็อบเจกต์และโครงสร้างข้อมูล

ภาคการศึกษาที่ 3 ปีการศึกษา 2567

สาขาวิชาเทคโนโลยีดิจิทัล กลุ่มหลักสูตรศาสตร์และศิลป์ดิจิทัล

มหาวิทยาลัยเทคโนโลยีสุรนารี

**รายละเอียดการแบ่งงานภายในกลุ่มที่ C2**

| ลำดับที่ | รหัสประจำตัว | ชื่อ - นามสกุล | หัวข้อที่รับผิดชอบ | ร้อยละที่ทำ |
|---|---|---|---|---|
| 10 | B6701611 | นายณรงค์ชัย ท่าทอง | 1. ออกแบบ GUI หน้าพนักงานเบื้องต้น<br>2. ระบบล็อกอิน<br>3. เมนูทั้งหมดหน้าพนักงาน<br>4. เมนูทั้งหมดหน้าเชฟ<br>5. รวมระบบทั้งหมดเข้าด้วยกัน | **30** |
| 16 | B6702076 | นางสาวณัฐกมล เพชร์รัตน์ | 1. ออกแบบ GUI หน้าเชฟเบื้องต้น<br>2. แผนภาพ Class Diagram<br>3. รูปเล่มรายงาน<br>4. ตรวจสอบและแก้ไขคำที่ใช้บนรายงาน | **20** |
| 28 | B6703257 | นายธณกร ศุภธรรมกิจ | 1. ออกแบบ GUI หน้าผู้จัดการเบื้องต้น<br>2. คลิปวีดิโอ และคู่มือการติดตั้งโปรแกรม<br>3. ตัดต่อคลิปวีดิโอนำเสนอ | **20** |
| 35 | B6703738 | นายเจษฎากร ทองสืบสาย | 1. ออกแบบ GUI หน้าผู้จัดการเบื้องต้น<br>2. ระบบหน้าผู้จัดการ<br>3. เมนูทั้งหมดของหน้าผู้จัดการ<br>4. คู่มือการใช้งานโปรแกรม | **30** |
| | | | **รวม (ร้อยละ)** | **100** |

# ระบบจัดการร้านอาหาร ( Restaurant System )

**ที่มาและความสำคัญ**

ในปัจจุบันธุรกิจร้านอาหารญี่ปุ่นได้รับความนิยมเป็นอย่างมาก การจัดการระบบภายในร้านอาหารให้มีประสิทธิภาพจึงเป็นสิ่งสำคัญมาก โดยเฉพาะการบริหารออเดอร์ เมนู และการให้บริการ การมีระบบจัดการร้านอาหารที่ดีจะช่วยให้การทำงานเป็นไปอย่างราบรื่น ลดข้อผิดพลาด และเพิ่มความพึงพอใจของลูกค้าได้

**วัตถุประสงค์**

1. เพื่อช่วยให้การจัดการร้านอาหารมีความเป็นระบบและมีประสิทธิภาพมากขึ้น
2. เพื่อเพิ่มความรวดเร็วในการทำงาน
3. เพื่อลดการสื่อสารผิดพลาดระหว่างคนรับออเดอร์กับคนทำอาหาร
4. เพื่อให้คณะผู้จัดทำฝึกทักษะในด้านการเขียนโปรแกรม
5. เพื่อจัดทำรายงานการสั่งซื้อและยอดขายของร้านอาหาร

ขอบเขตของโครงงาน

1. ระบบสามารถรองรับการสั่งอาหารของลูกค้าได้
2. มีระบบจัดการเมนูอาหารที่สามารถเพิ่ม ลด และแก้ไขได้
3. ระบบสามารถแสดงสถานะของออเดอร์ลูกค้าได้แบบเรียลไทม์
4. รองรับการชำระเงินที่สามารถคำนวณเงินทอนได้
5. ผู้ใช้งานแบ่งเป็น 3 กลุ่ม ได้แก่ พนักงานผู้ให้บริการรับออเดอร์ พนักงานทำอาหาร และผู้จัดการ

**ประโยชน์ที่คาดว่าจะได้รับ**

1. สามารถจัดการข้อมูลได้อย่างเป็นระบบมากขึ้น
2. ลดข้อผิดพลาดในการสั่งอาหาร และเพิ่มประสิทธิภาพในการทำงาน
3. เพื่อให้ลูกค้าสามารถสั่งอาหารผ่านระบบดิจิทัลได้อย่างสะดวก
4. สรุปข้อมูลการสั่งอาหารแต่ละวัน รวมถึงยอดขาย เพื่อช่วยในการวิเคราะห์และปรับปรุงการดำเนินงานของร้าน

**เมนูหลักของระบบ** มีดังนี้

1. ระบบล็อคอิน (Login System)
2. เมนูรับออเดอร์ลูกค้า (เมนูสำหรับพนักงาน) (Accepting orders Menu)
3. เมนูแก้ไขออเดอร์ (เมนูสำหรับพนักงาน) (Check Orders Menu)
4. เมนูเรียกเก็บเงิน (เมนูสำหรับพนักงาน) (Payment Menu)
5. เมนูอัพเดตสถานะและดูออเดอร์ (เมนูสำหรับเชฟ) (Order status update and View Menu)
6. เมนูดูคิวออเดอร์ (เมนูสำหรับเชฟ) (Queue Menu)
7. เมนูรายชื่อพนักงาน (เมนูสำหรับผู้จัดการ) (Employee list Menu)
8. เมนูเพิ่มพนักงาน (เมนูสำหรับผู้จัดการ) (Add Employee Menu)
9. เมนูเพิ่มรายการอาหาร (เมนูสำหรับผู้จัดการ) (Add Menu)
10. เมนูรายการอาหารทั้งหมด (เมนูสำหรับผู้จัดการ) (Menu List)
11. เมนูรายงานยอดขาย (เมนูสำหรับผู้จัดการ) (daily Report Menu)

1. ระบบล็อคอิน (Login System)
   - การล็อคอินเข้าสู่ระบบ ผู้ใช้งานต้องระบุ Username, Password เพื่อเข้าใช้งาน
   - โดย Username ของแต่ละหน้าที่จะต่างกันออกไป คือ

     1 - 10 จะเป็นของผู้จัดการ

     11 - 20 จะเป็นของพนักงงานทั่วไป

     21 - 30 จะเป็นของเชฟ

     และสามารถตั้ง Password เองได้

2. เมนูรับออเดอร์ลูกค้า (Accepting orders Menu)
   - เป็นเมนูที่พนักงานรับออเดอร์สามารถใช้ในการเพิ่มและรับออเดอร์จากลูกค้า
   - โดยจะระบุเลขโต๊ะของลูกค้า รายการค้นหาเมนูอาหารที่ลูกค้าต้องการ แล้วเพิ่มในรายการของลูกค้าโต๊ะนั้น
   - สามารถเพิ่มหมายเหตุพิเศษในออเดอร์ได้ เช่น ไม่เผ็ด หรือ เพิ่มซอส
   - เมื่อเสร็จแล้วระบบจะส่งรายการคำสั่งซื้อไปที่ระบบดูออเดอร์ (สำหรับเชฟ)

3. เมนูแก้ไขออเดอร์ (เมนูสำหรับพนักงาน) (Check Orders Menu)

- เป็นเมนูที่พนักงานรับออเดอร์สามารถแก้ไขข้อมูลออเดอร์ปัจจุบันของลูกค้าโต๊ะนั้น ๆ ได้ โดยจะสามารถเพิ่มเมนูใหม่ หรือลบเมนูเดิม
- หากเชฟเริ่มทำเมนูนั้นแล้ว จะไม่สามรถลบเมนูที่ต้องการแก้ไขได้
- ระบบจะอัพเดตข้อมูลเมื่อกดอัพเดต แล้วแจ้งไปที่เชฟเพื่อทราบการเปลี่ยนแปลง

4. เมนูเรียกเก็บเงิน (เมนูสำหรับพนักงาน) (Payment Menu)
    - เป็นเมนูที่คำนวณยอดเงินที่ต้องชำระ หลังจากทานอาหารเสร็จ หรือ เรียกเช็คบิล ระบบจะแสดงรายการออเดอร์ และ อาหารที่ลูกค้าสั่งในออเดอร์ พร้อมจำนวนออเดอร์ ราคาอาหาร ยอดรวมสุทธิ
    - หากลูกค้าชำระด้วยเงินสด พนักงานจะต้องระบุจำนวนเงินที่รับมา จากนั้นระบบจะคำนวณเงินทอนให้อัตโนมัติ

5. เมนูอัพเดตสถานะและดูออเดอร์ (เมนูสำหรับเชฟ) (Order status update and View Menu)
    - เชฟสามารถดูรายการออเดอร์ที่ต้องทำในระบบได้ โดยระบบจะแสดงข้อมูลออเดอร์คือ เลขที่โต๊ะ ชื่อเมนูอาหารที่ต้องทำ หมายเหตุพิเศษ จำนวนที่สั่ง ราคารวมทั้งหมด
      ปุ่ม Finish (เมื่อทำอาหารเสร็จแล้ว) และ Delete (ไม่มีวัตถุดิบ) เพื่อแจ้งสถานะอาหารให้พนักงานทราบ
    - เชฟสามารถ อัพเดตสถานะของออเดอร์ได้ เช่น

        สีม่วง หมายถึง ไม่มีออเดอร์

        สีแดง หมายถึง เชฟรับออเดอร์แล้ว และกำลังทำอาหาร

        สีเขียว หมายถึง อาหารพร้อมเสิร์ฟ ระบบจะแจ้งให้พนักงานเสิร์ฟมารับอาหารได้

6. เมนูดูคิวออเดอร์ (เมนูสำหรับเชฟ) (Queue Menu)
    - เชฟสามารถดูคิวของเดอร์ที่สั่งของแต่ละโต๊ะได้ โดยระบบจะแสดงข้อมูล เลขที่โต๊ะ และเวลาที่สั่ง
    - เมื่อกดเข้าไปจะแสดงข้อมูลออเดอร์คือ เลขที่โต๊ะ ชื่อเมนูอาหารที่ต้องทำ หมายเหตุพิเศษ จำนวนที่สั่ง ราคารวมทั้งหมดปุ่ม Finish (เมื่อทำอาหารเสร็จแล้ว) และ Delete (เมื่อไม่มีวัตถุดิบ) เพื่อแจ้งสถานะอาหารให้พนักงานทราบ

7. เมนูรายชื่อพนักงาน (เมนูสำหรับผู้จัดการ) (Employee list Menu)
    - เป็นเมนูที่ใช้แสดงรายชื่อพนักงานทั้งหมดในร้านอาหาร พร้อมข้อมูลที่สำคัญ เช่น ลำดับที่ ชื่อผู้ใช้ ชื่อ นามสกุล ตำแหน่ง และ รหัสผ่าน (ที่พนักงาานคนนั้น ๆ สร้างขึ้น)
    - สามารถค้นหา และลบข้อมูลพนักงานได้

8. เมนูเพิ่มพนักงาน (เมนูสำหรับผู้จัดการ) (Add Employee Menu)
    - เป็นเมนูไว้สำหรับเพิ่มพนักงานใหม่ โดยจะต้องกรอกข้อมูลลำดับที่ ชื่อผู้ใช้ ชื่อ นามสกุล ตำแหน่ง และ รหัสผ่าน (ที่พนักงาานคนนั้น ๆ สร้างขึ้น)

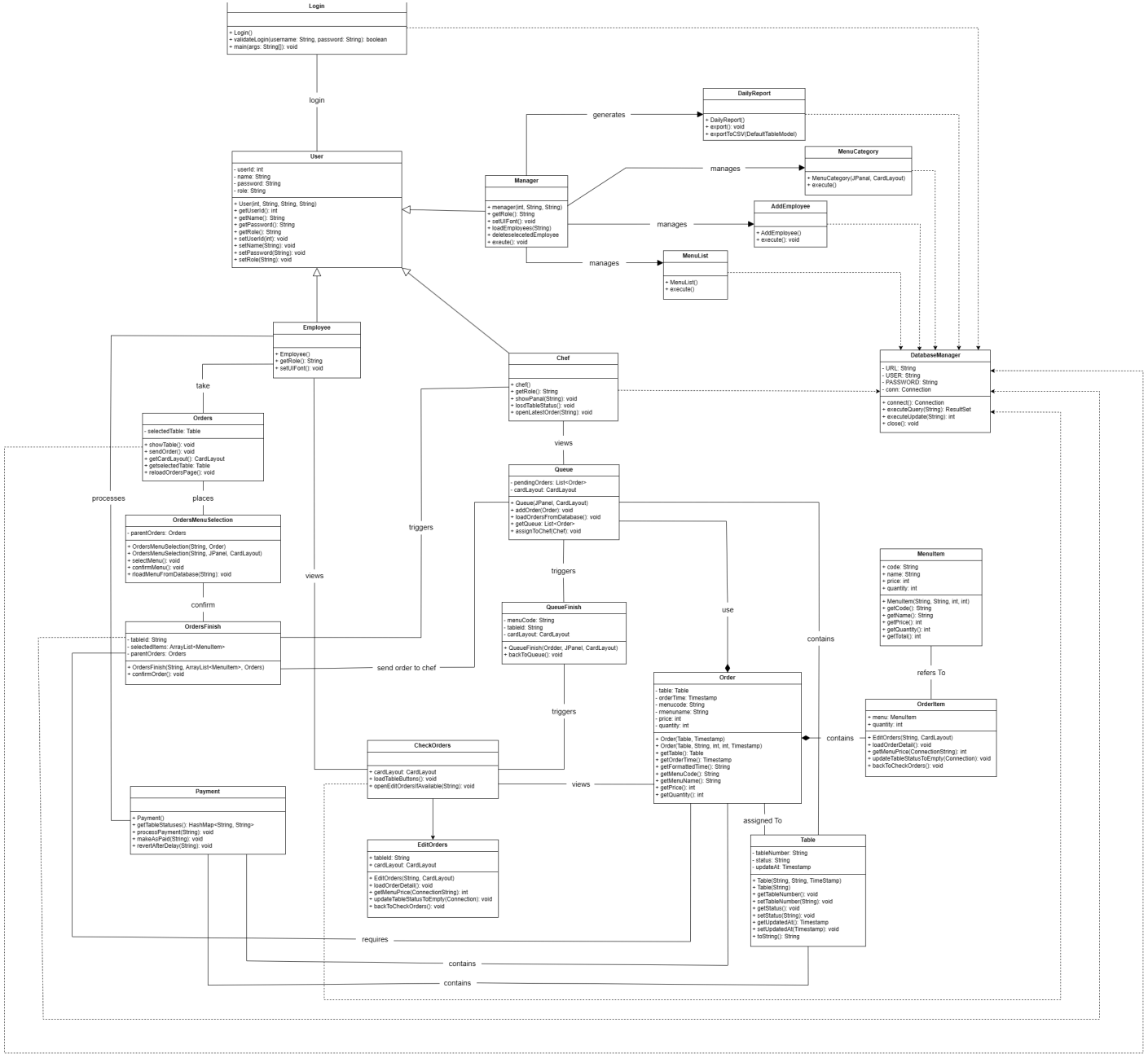
9. เมนูเพิ่มรายการอาหาร (เมนูสำหรับผู้จัดการ) (Add Menu)
    - เป็นเมนูไว้ใช้สำหรับเพิ่มเมนูอาหารใหม่ โดยจะต้องกรอกข้อมูล รหัสของอาหาร ชื่อเมนู และราคา

10. เมนูรายการอาหารทั้งหมด (เมนูสำหรับผู้จัดการ) (Menu List)
    - เป็นเมนูที่จะแสดง รหัสของอาหาร ชื่อเมนู และราคา ของอาหารทั้งหมดในร้าน
    - สามารถลบ และแก้ไขชื่อ และราคาของเมนูอาหารที่มีอยู่แล้วได้

11. เมนูรายงานยอดขายรายวัน (เมนูสำหรับผู้จัดการ) (daily Report Menu)
    - เป็นเมนูที่สามารถคำนวณยอดรวมยอดขายที่ทำได้ทั้งหมดของวัน ไม่สามารถดูย้อนหลังของวันก่อนหน้าได้
    - โดยจะแสดงข้อมูลเป็นกราฟว่าขายเมนูอะไรได้เป็นปริมาณเท่าไหร่ และตารางแสดงรายการเมนูอาหาร ราคา จำนวนที่สั่ง ยอดรวม และหมายเหตุพิเศษต่าง ๆ ที่ขายได้

- สามรถนำข้อมูลยอดขายออกมาเป็นไฟล์ (.CSV) ได้ เพื่อเก็บไว้สำหรับการวิเคราะห์ข้อมูลยอดขายต่อไป

[Class Diagram](#) (คลิกที่คำว่า Class Diagram เพื่อดูรูปเต็ม)



**คู่มือการติดตั้งโปรแกรม ระบบจัดการร้านอาหาร (Restaurant Systems)**

ลิ้งค์โหลด xampp : คลิ๊กตรงนี้

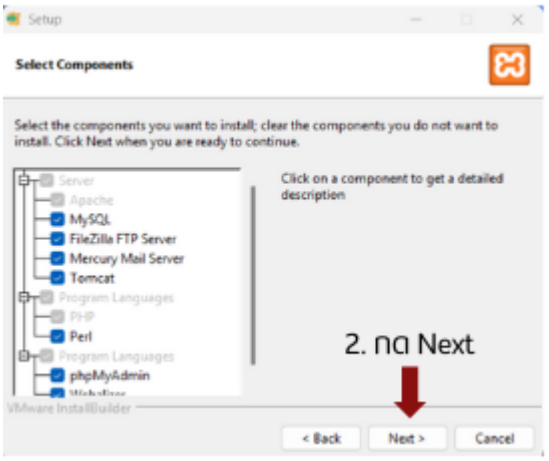
ลิ้งค์โหลด java : คลิ๊กตรงนี้

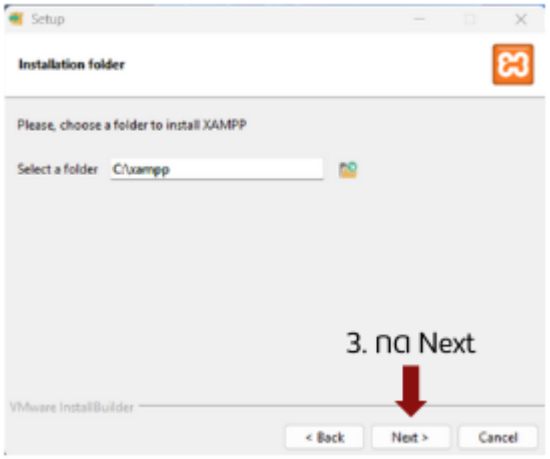
ลิ้งค์โหลดไฟล์สำหรับติดตั้งโปรแกรม : คลิ๊กตรงนี้

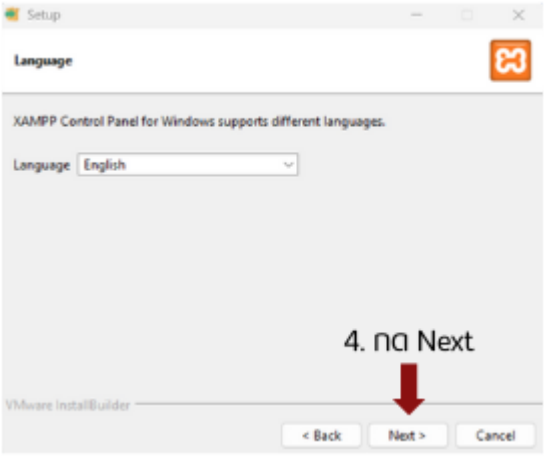
ลิ้งค์สอนติดตั้งโปรแกรมใน Eclipse : คลิ๊กตรงนี้

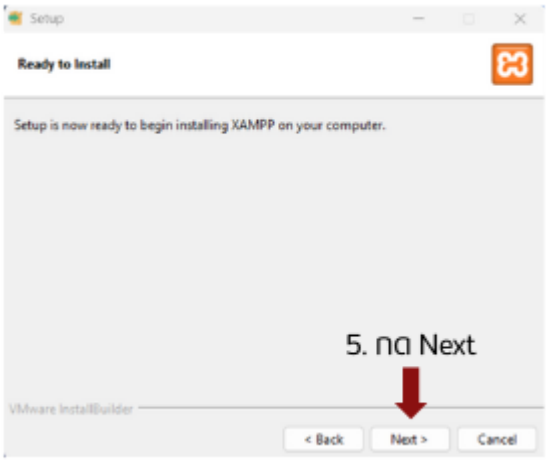
1. ติดตั้งโปรแกรม xampp

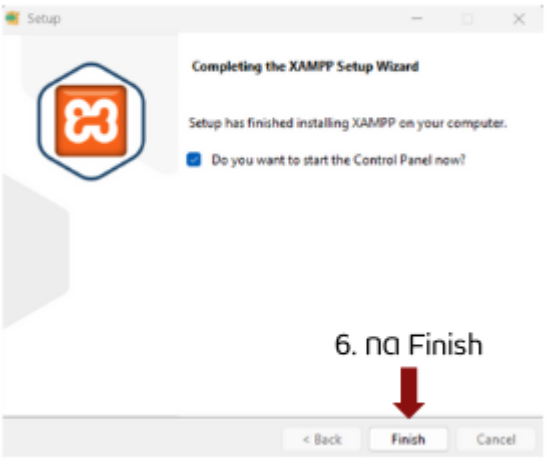


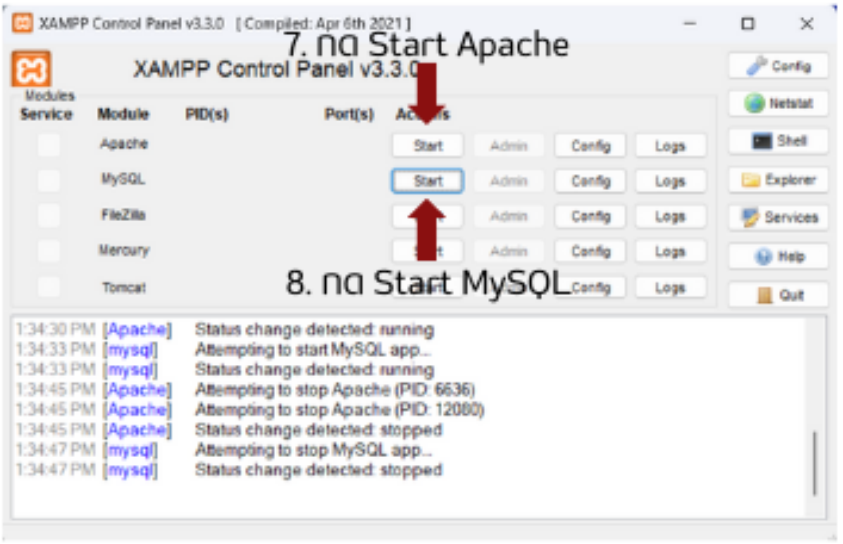



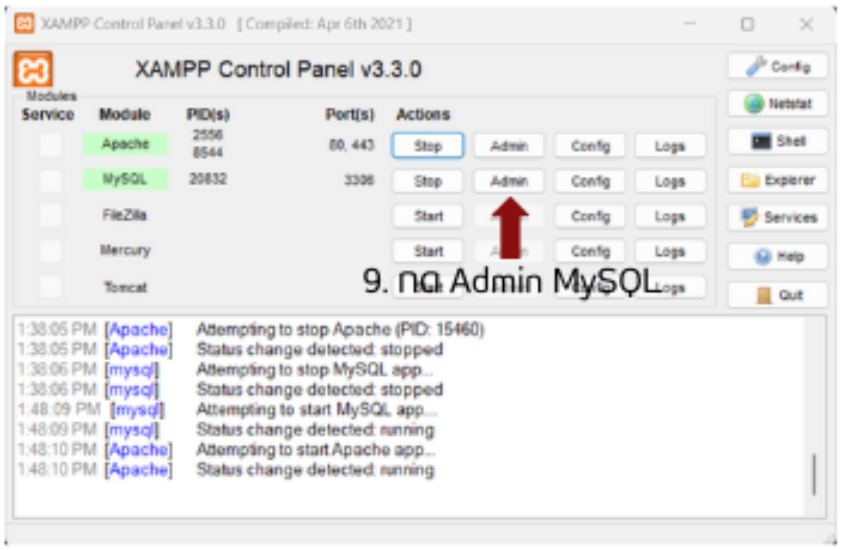



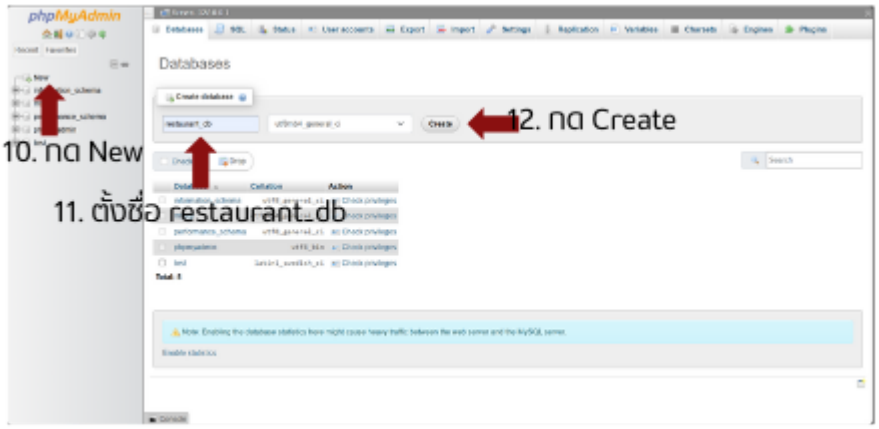



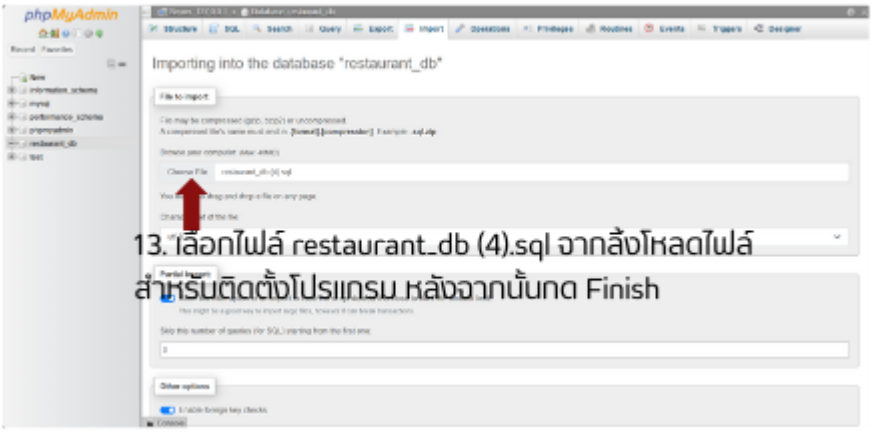

Setup
Ready to Install
Setup is now ready to begin installing XAMPP on your computer.
5. na Next
VMware InstallBuilder
< Back
Next >
Cancel


Setup
Completing the XAMPP Setup Wizard
Setup has finished installing XAMPP on your computer.
Do you want to start the Control Panel now?
6. na Finish
< Back
Finish
Cancel


XAMPP Control Panel v3.3.0 [ Compiled: Apr 6th 2021 ]
7. na Start Apache
XAMPP Control Panel v3.3.0
Modules
Service
Module
PID(s)
Port(s)
Actions
Apache
Start
Admin
Config
Logs
MySQL
Start
Admin
Config
Logs
FileZilla
Admin
Config
Logs
Mercury
Admin
Config
Logs
Tomcat
Config
Logs
8. na Start MySQL
Config
Netstat
Shell
Explorer
Services
Help
Quit
1:34:30 PM [Apache] Status change detected: running
1:34:33 PM [mysql] Attempting to start MySQL app...
1:34:33 PM [mysql] Status change detected: running
1:34:45 PM [Apache] Attempting to stop Apache (PID: 6636)
1:34:45 PM [Apache] Attempting to stop Apache (PID: 12080)
1:34:45 PM [Apache] Status change detected: stopped
1:34:47 PM [mysql] Attempting to stop MySQL app...
1:34:47 PM [mysql] Status change detected: stopped


XAMPP Control Panel v3.3.0 [ Compiled: Apr 6th 2021 ]
XAMPP Control Panel v3.3.0
Modules
Service
Module
PID(s)
Port(s)
Actions
Apache
2556
8544
80, 443
Stop
Admin
Config
Logs
MySQL
20832
3306
Stop
Admin
Config
Logs
FileZilla
Start
Config
Logs
Mercury
Start
Config
Logs
Tomcat
Config
Logs
9. na Admin MySQL
Config
Netstat
Shell
Explorer
Services
Help
Quit
1:38:05 PM [Apache] Attempting to stop Apache (PID: 15460)
1:38:05 PM [Apache] Status change detected: stopped
1:38:06 PM [mysql] Attempting to stop MySQL app...
1:38:06 PM [mysql] Status change detected: stopped
1:48:09 PM [mysql] Attempting to start MySQL app...
1:48:09 PM [mysql] Status change detected: running
1:48:10 PM [Apache] Attempting to start Apache app...
1:48:10 PM [Apache] Status change detected: running


Databases
10. กด New
11. ตั้งชื่อ restaurant_db
12. กด Create


Importing into the database "restaurant_db"
13. เลือกไฟล์ restaurant_db (4).sql จากลิ้งโหลดไฟล์
สำหรับติดตั้งโปรแกรม หลังจากนั้นกด Finish

2. ติดตั้ง java



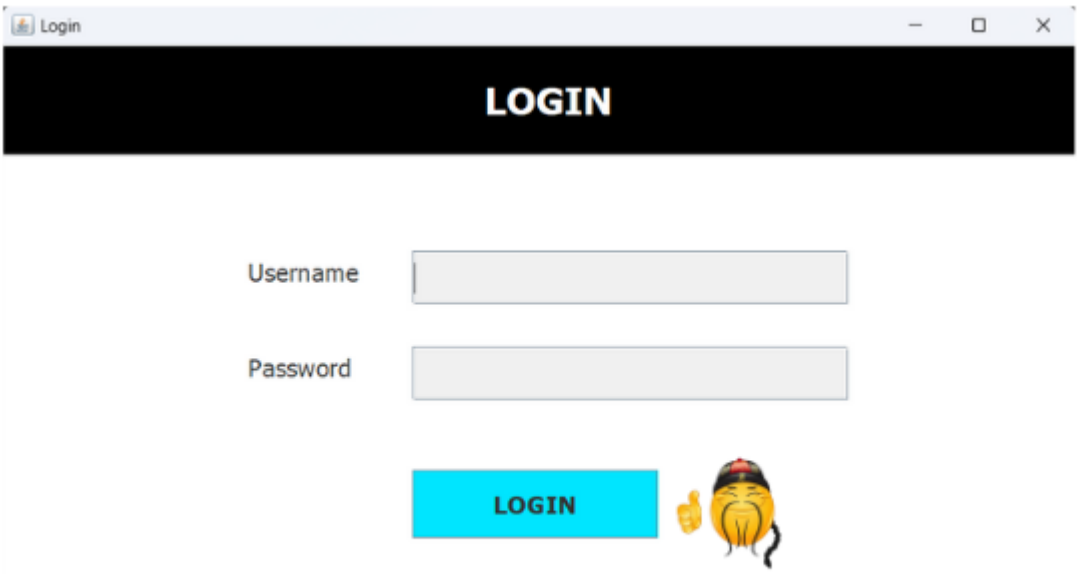






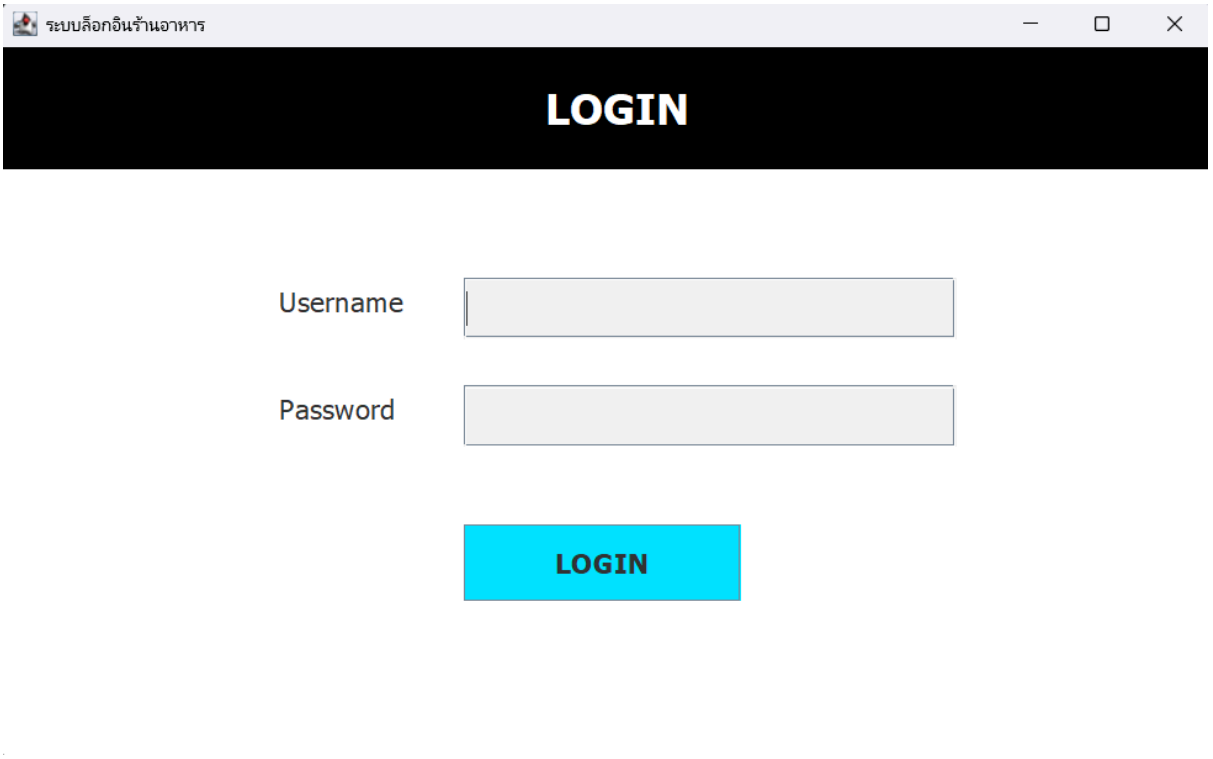

3. โหลด RestaurantApp.exe จากลิ้งโหลดไฟล์สำหรับติดตั้งโปรแกรม

และกดรันโปรแกรม



**คู่มือการใช้งานโปรแกรม ระบบจัดการร้านอาหาร (Restaurant Systems)**

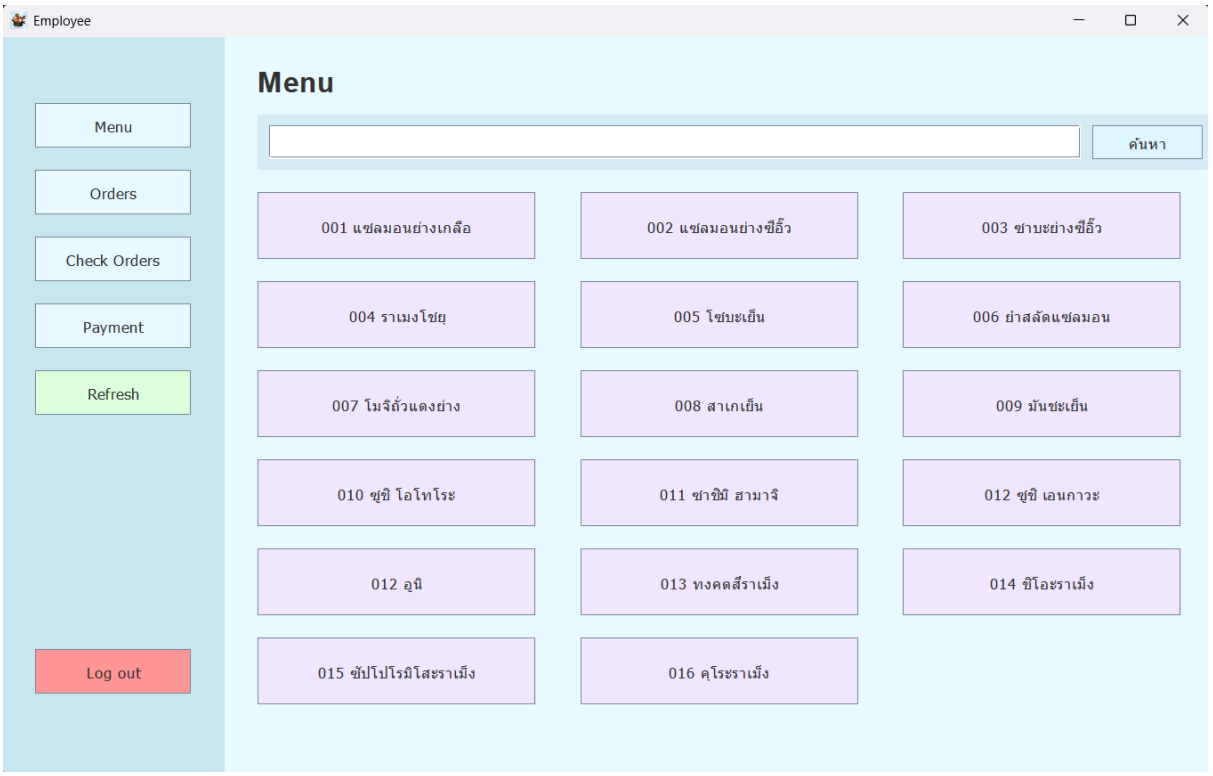
1. หน้า Log in




ภาพที่ 1 หน้า login

คำอธิบาย : ในหน้า log in ให้กรอก username, password ให้เรียบร้อย หลังจากนั้นให้กดปุ่ม LOGIN เพื่อเข้าใช้งาน

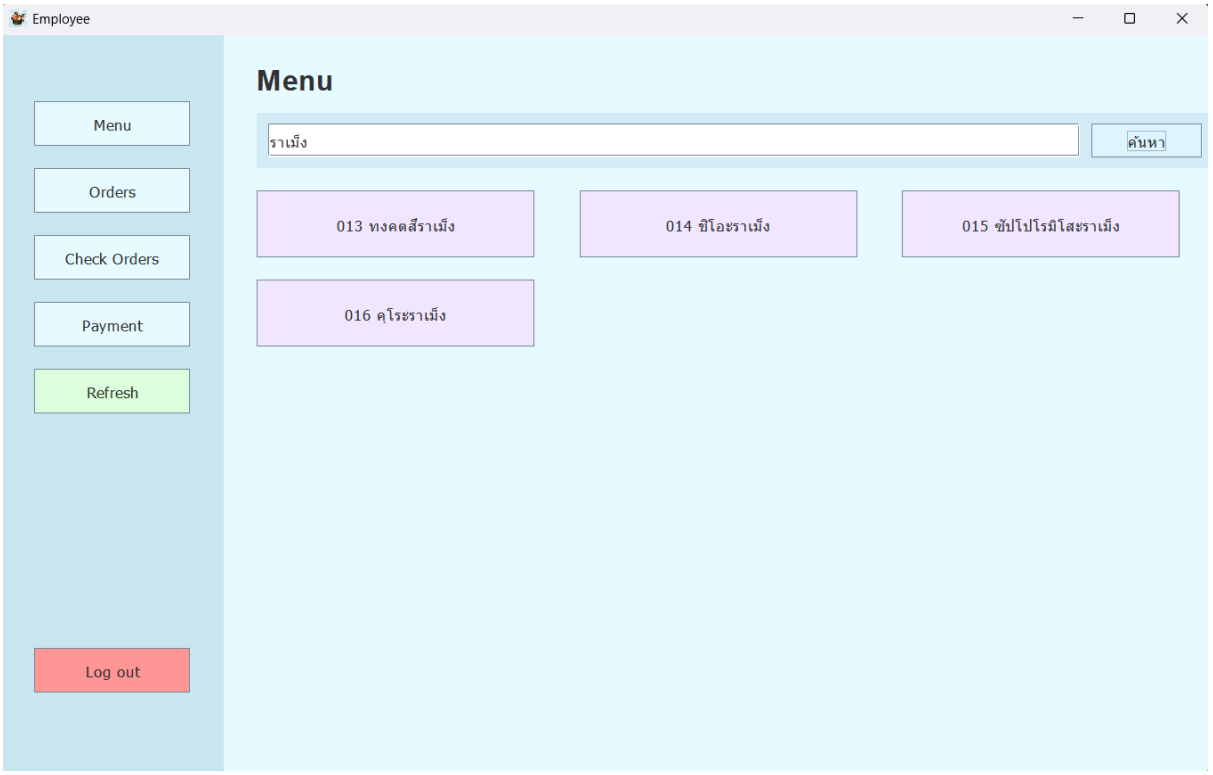
2.หน้าต่างเมนูของ พนักงาน




ภาพที่ 2 หน้าต่างเมนูของ พนักงาน

คำอธิบาย : ในกรณีที่เข้าสู่ระบบด้วยตำแหน่งพนักงาน โปรแกรมจะแสดงหน้าต่างดังภาพ

2.1 หน้าต่างรายการอาหาร (Menu)

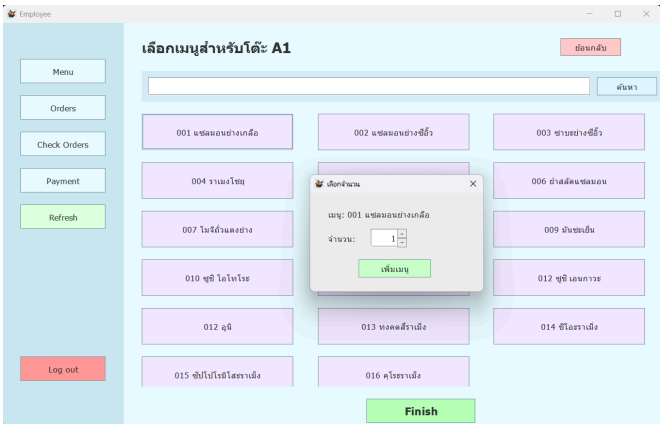



ภาพที่ 3 หน้าต่างรายการอาหาร (Menu)

คำอธิบาย : เมื่อผู้ใช้กดที่ ช่องค้นหา จะสามารถค้นหาได้ เมื่อกด ปุ่มค้นหา โปรแกรมจะแสดงรายการอาหารดังภาพ

2.2 หน้าต่างการสั่งอาหาร (Orders)



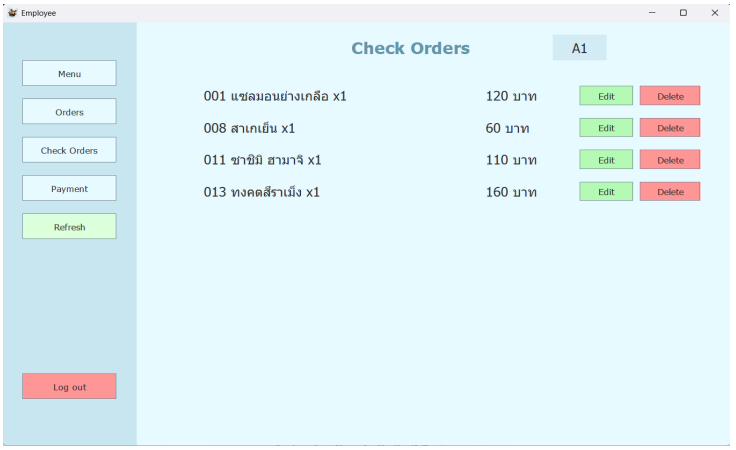




ภาพที่ 4 , 5

คำอธิบาย : ในการสั่งอาหาร ให้ผู้ใช้กดที่ปุ่ม Orders ในแถบด้านขวาจะแสดงหน้าต่างดังภาพที่ 4 จากนั้นให้ทำการเลือกโต๊ะ เมื่อเลือกโต๊ะเสร็จสิ้น โปรแกรมจะแสดงหน้าต่างดังภาพที่ 5 ให้ผู้ใช้เลือกอาหารและใส่จำนวนให้เรียบร้อย หลังจากนั้นกดเพิ่มเมนูเป็นอันเสร็จ

2.3 หน้าต่างการตรวจสอบและแก้ไขการสั่งอาหาร (Check Orders)

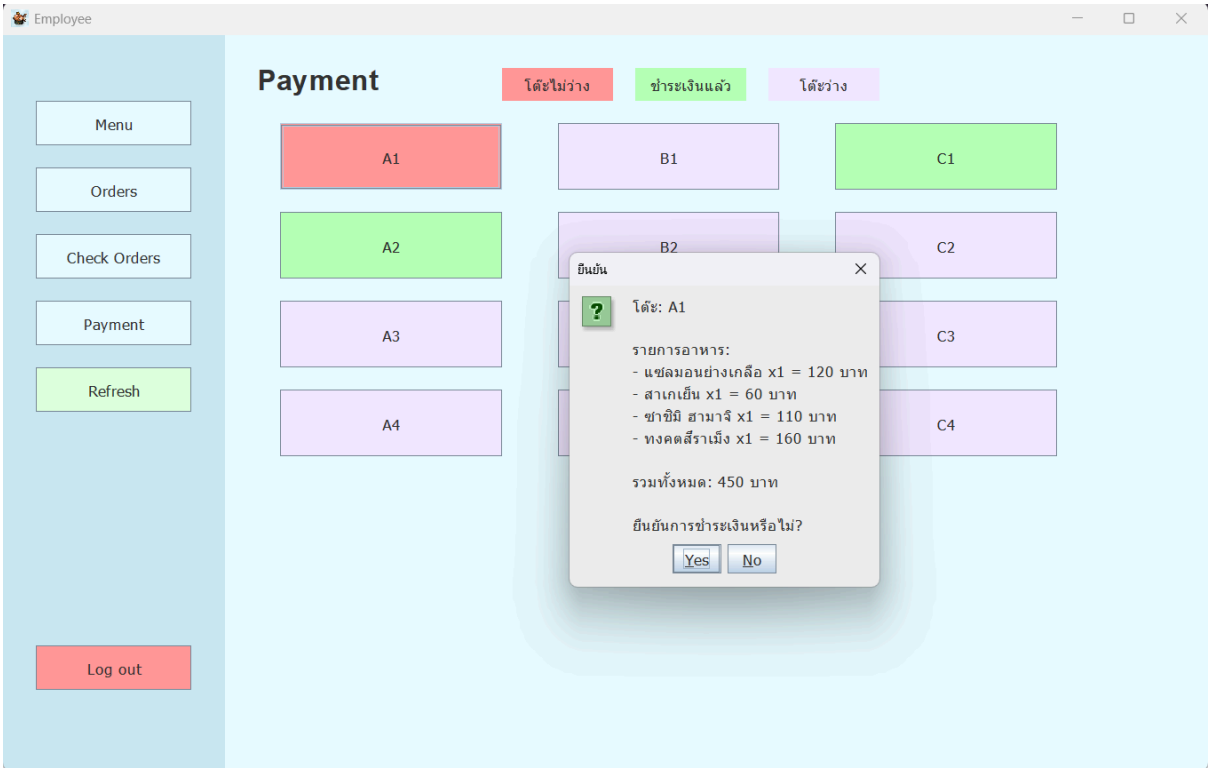



ภาพที่ 6 , 7

คำอธิบาย : ผู้ใช้สามารถเช็กออเดอร์ ได้โดยกดที่ปุ่ม Check orders โปรแกรมจะแสดงหน้าต่างดังภาพที่ 6 ให้ทำการเลือกโต๊ะที่จะเช็ก เมื่อเลือกโต๊ะแล้ว โปรแกรมจะแสดงหน้าต่างดังภาพที่ 7 ในหน้านี้จะสามารถแก้ไข , ลบ เมนูที่สั่งไปแล้วได้โดยการกดที่ปุ่มดังกล่าว

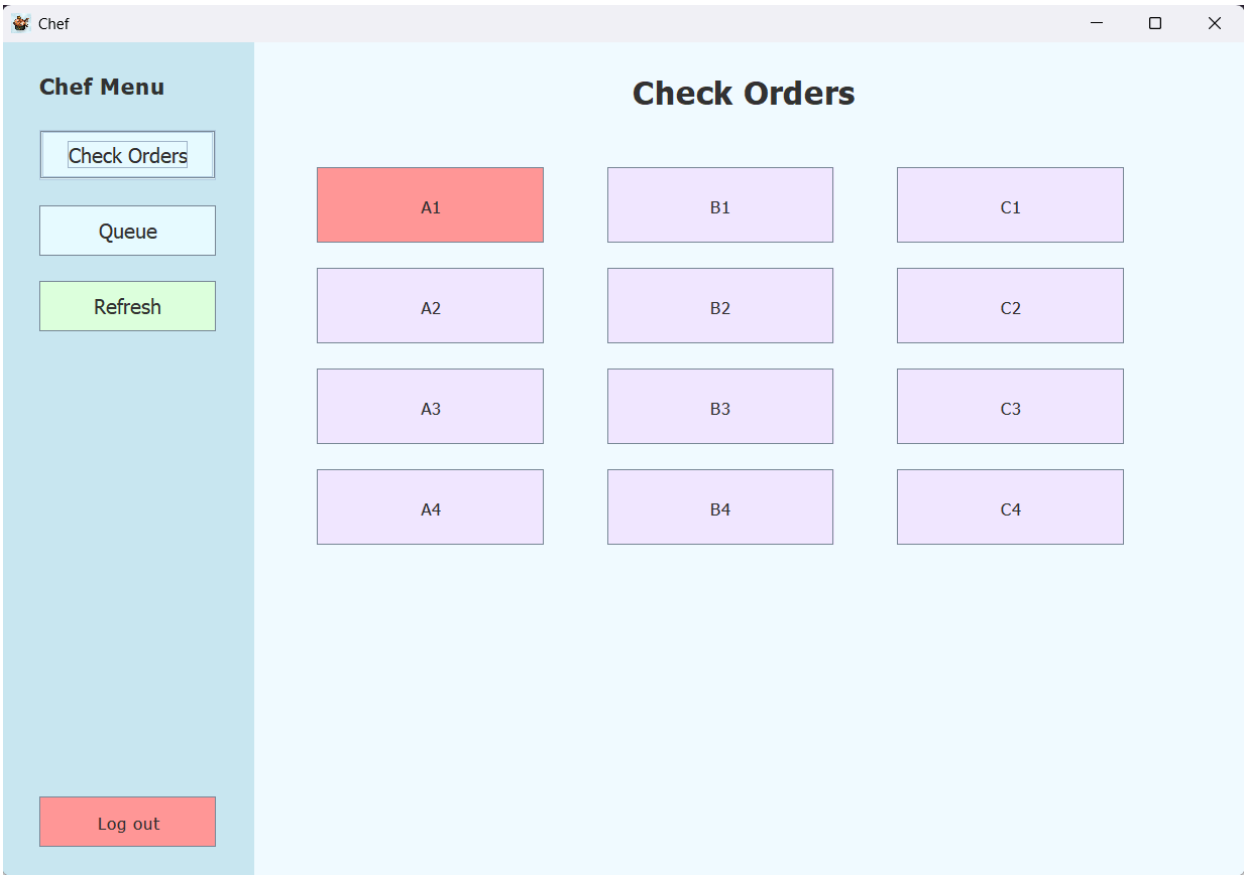
2.4 หน้าต่างการชำระเงิน (Payment)




ภาพที่ 8

คำอธิบาย : ผู้ใช้สามารถทำการคิดเงินได้โดยการกดที่ปุ่ม Payment เมื่อลูกค้าทำการชำระเงินแล้ว และเช็กจำนวนเรียบร้อย ให้ผู้ใช้กดปุ่ม Yes เพื่อทำการยืนยันการชำระเงิน เป็นอันเสร็จสิ้น

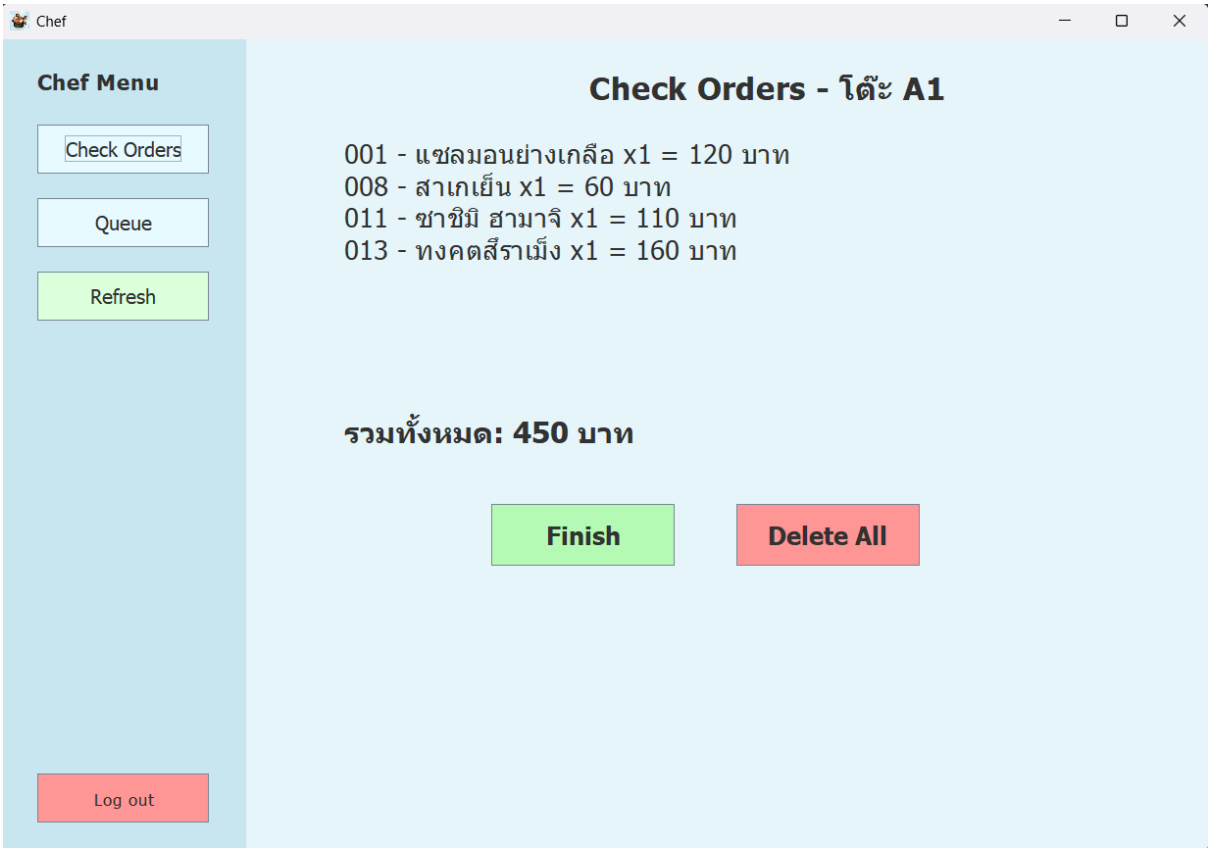
3. หน้าต่างเมนูของ เชฟ




ภาพที่ 9

คำอธิบาย : ในกรณีที่เข้าสู่ระบบด้วยตำแหน่งเชฟ โปรแกรมจะแสดงโต๊ะ ดังภาพที่ 9 โดยให้ผู้ใช้เลือกโต๊ะที่จะทำการเช็กออเดอร์

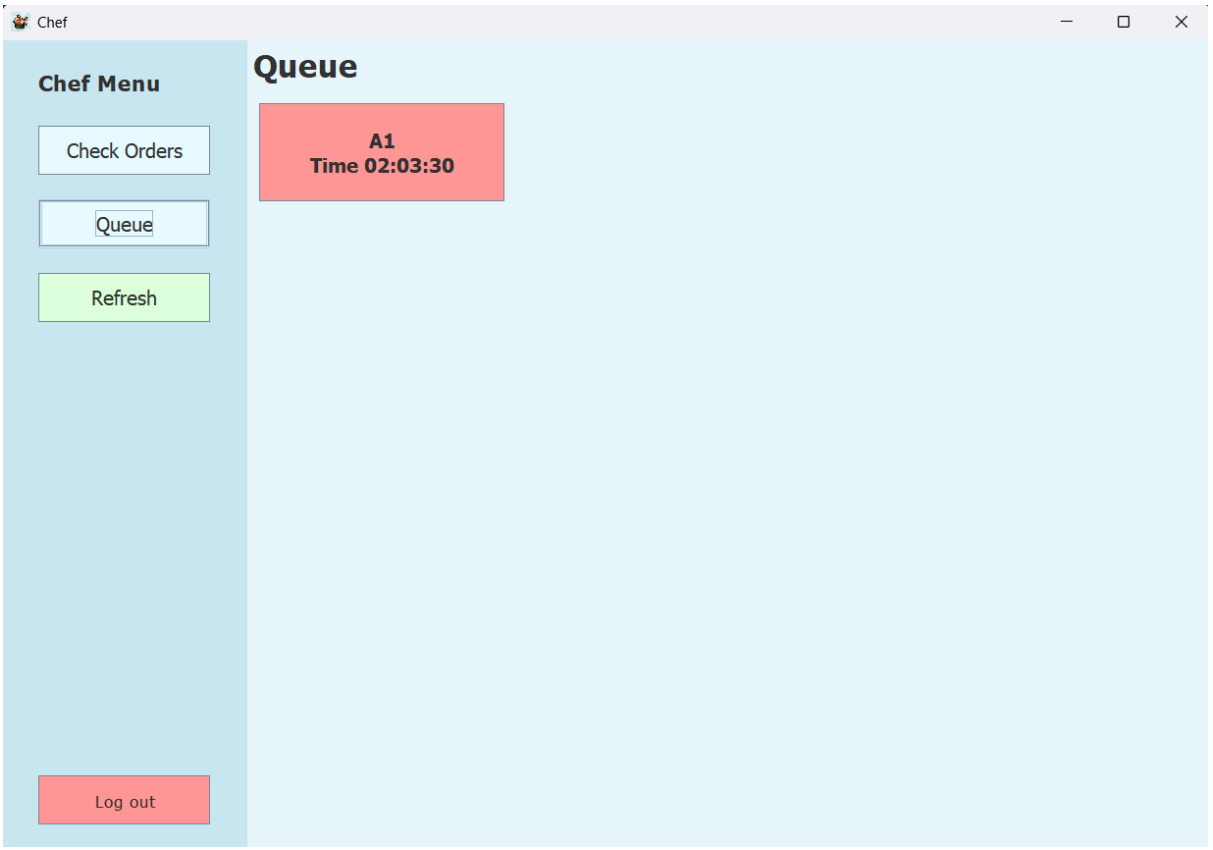
3.1 หน้าต่างดูโต๊ะที่สั่งอาหาร (Check Orders)




ภาพที่ 10

คำอธบาย : เมื่อทำการเลือกโต๊ะเสร็จโปรแกรมจะแสดงรายละเอียดของ ออเดอร์ของโต๊ะที่เลือก ดังภาพที่ 10

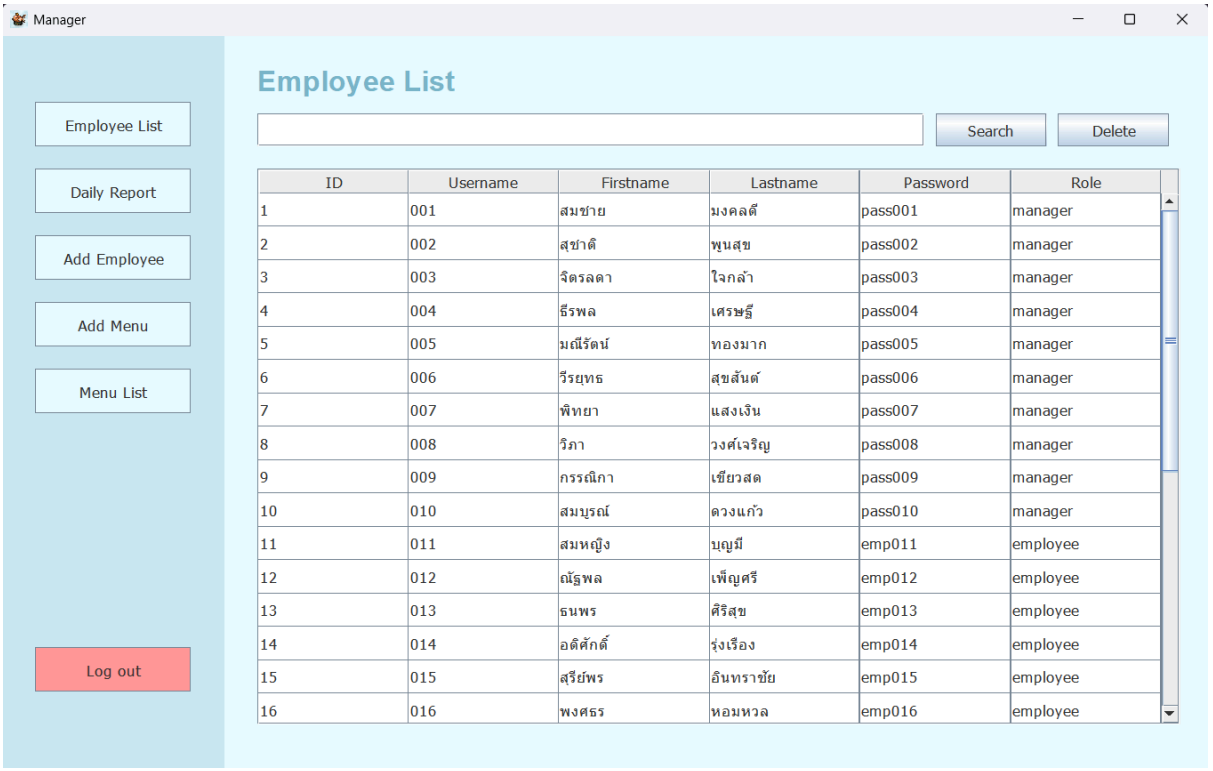
3.2 หน้าต่างสำหรับดูลำดับคิวที่สั่ง (Queue)




ภาพที่ 11

คำอธบาย : ผู้ใช้สามารถเช็กคิวได้โดยการกดที่ปุ่ม Queue แล้ว โปรแกรมจะแสดงหน้าต่างดังภาพที่ 11

4. หน้าต่างเมนูของ ผู้จัดการ

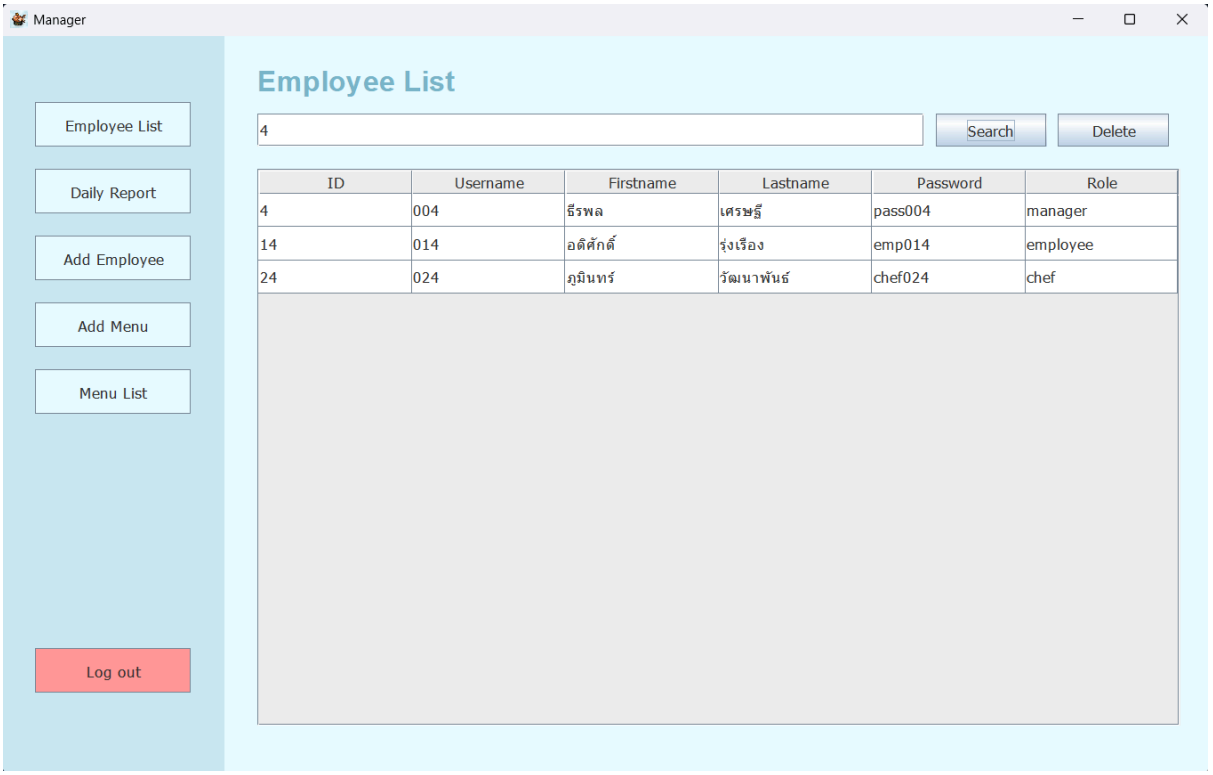



| ID | Username | Firstname | Lastname | Password | Role |
|---|---|---|---|---|---|
| 1 | 001 | สมชาย | มงคลดี | pass001 | manager |
| 2 | 002 | สุชาติ | พูนสุข | pass002 | manager |
| 3 | 003 | จิตรลดา | ใจกล้า | pass003 | manager |
| 4 | 004 | ธีรพล | เศรษฐี | pass004 | manager |
| 5 | 005 | มณีรัตน์ | ทองมาก | pass005 | manager |
| 6 | 006 | วีรยุทธ | สุขสันต์ | pass006 | manager |
| 7 | 007 | พิทยา | แสงเงิน | pass007 | manager |
| 8 | 008 | วิภา | วงศ์เจริญ | pass008 | manager |
| 9 | 009 | กรรณิกา | เขียวสด | pass009 | manager |
| 10 | 010 | สมบูรณ์ | ดวงแก้ว | pass010 | manager |
| 11 | 011 | สมหญิง | บุญมี | emp011 | employee |
| 12 | 012 | ณัฐพล | เพ็ญศรี | emp012 | employee |
| 13 | 013 | ธนพร | ศิริสุข | emp013 | employee |
| 14 | 014 | อดิศักดิ์ | รุ่งเรือง | emp014 | employee |
| 15 | 015 | สุรีย์พร | อินทราชัย | emp015 | employee |
| 16 | 016 | พงศธร | หอมหวล | emp016 | employee |

ภาพที่ 12

คำอธบาย : ในกรณีเข้าสู่ระบบด้วยตำแหน่งผู้จัดการ โปรแกรมจะทำการแสดงรายชื่อพนักงานทั้งหมด ดังภาพที่ 12

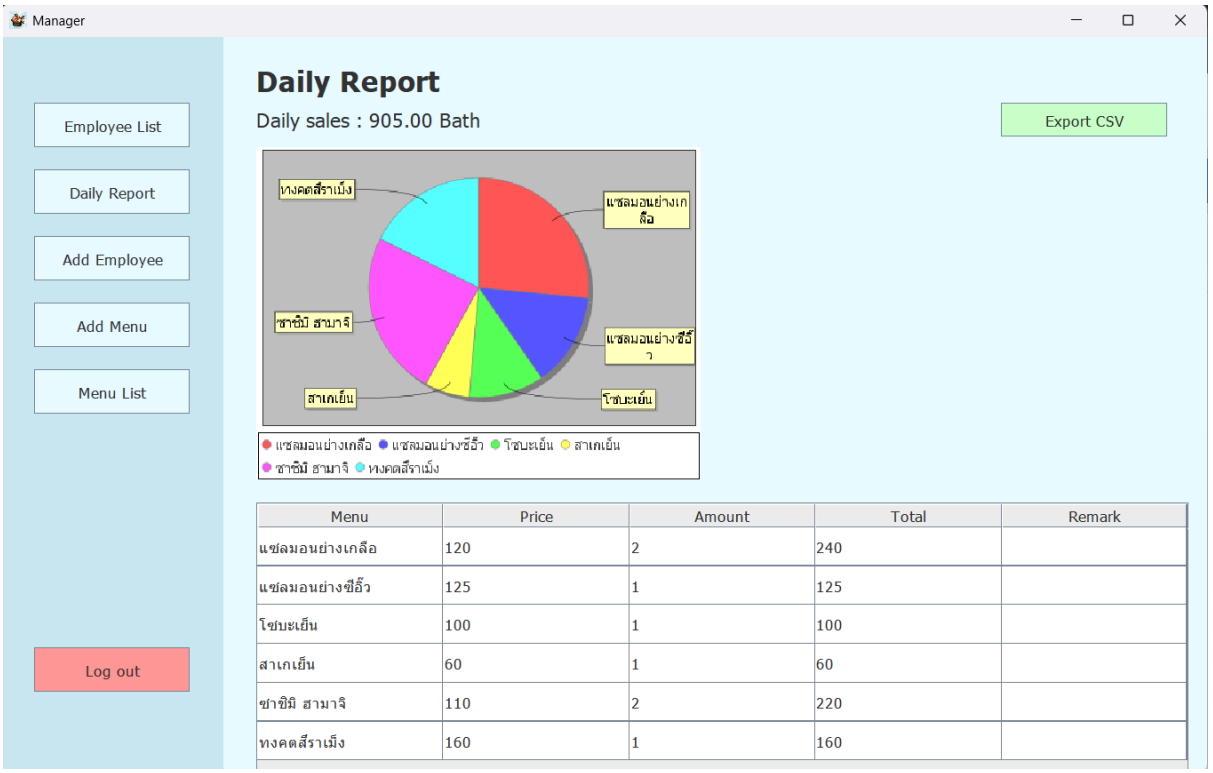
4.1 หน้าต่างแสดงรายชื่อพนักงานที่ค้นหาทั้งหมด (Employee List)




ภาพที่ 13

คำอธบาย : ผู้ใช้สามารถค้นหารายชื่อพนักงานได้โดยกดที่ช่องค้นหา แล้วกรอกข้อมูลของพนักงาน เช่น ชื่อ , รหัส , ตำแหน่ง จากนั้นให้กดปุ่มค้นหา ในตารางจะแสดงข้อมูลของพนักงานที่ค้นหาเจอ

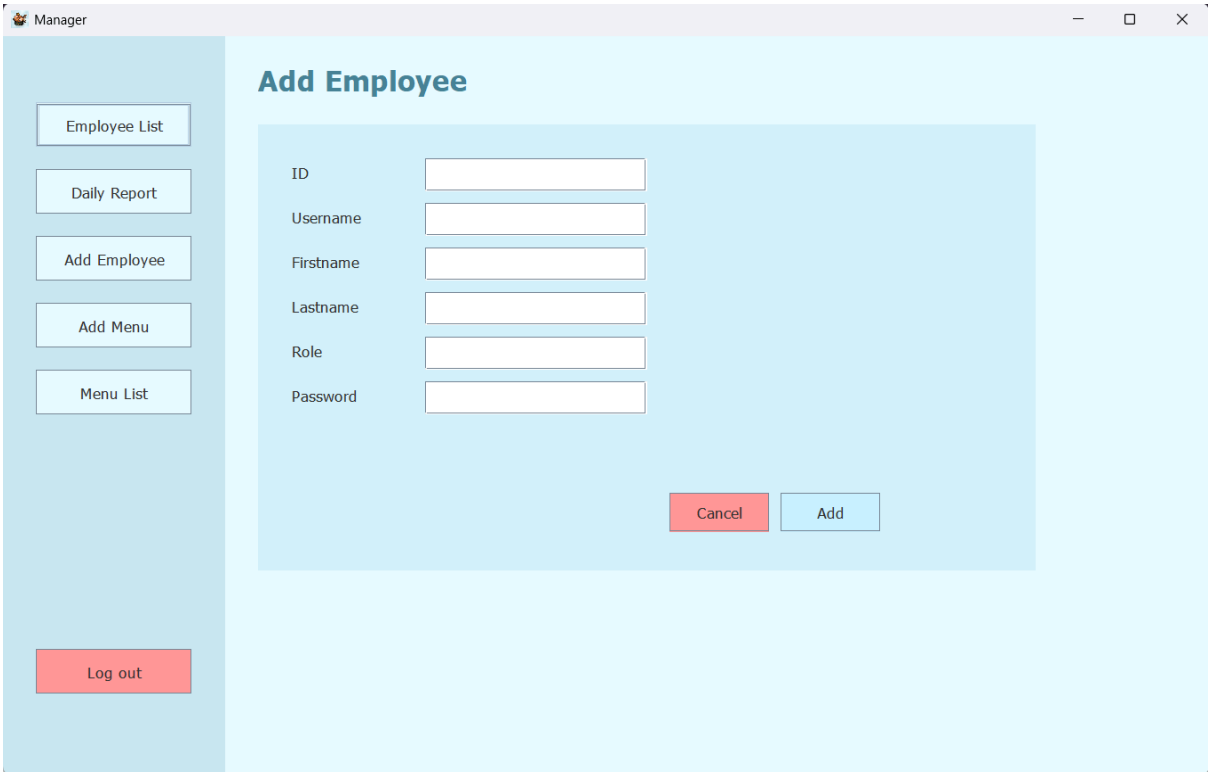
4.2 หน้าต่างดูรายงานยอดขายประจำวัน (Daily Report)




ภาพที่ 14

คำอธบาย : ผู้ใช้สามารถดูรายงานประจำวันได้ โดยกดที่ปุ่ม Daily Report โปรแกรมจะแสดงหน้าต่าง ดังภาพ 14 และ ผู้ใช้

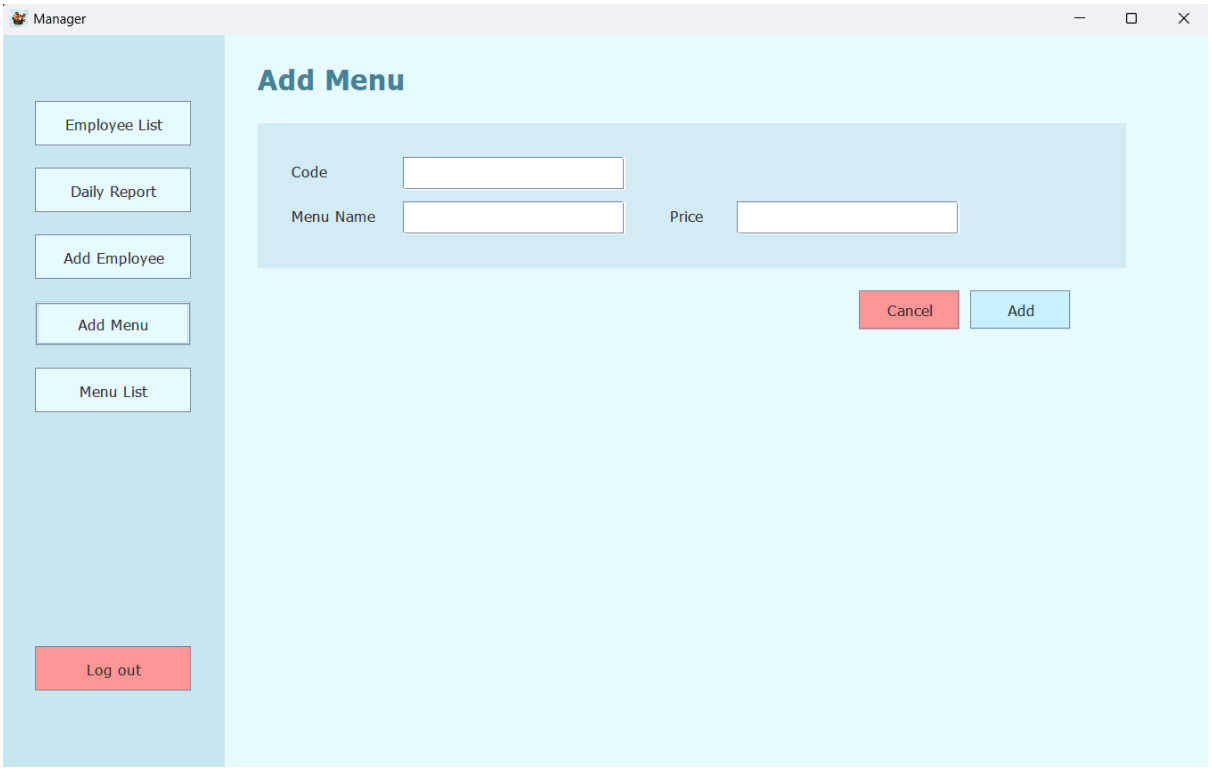
4.3 หน้าต่างเพิ่มพนักงาน (Add Employee)




ภาพที่ 15

คำอธบาย : ผู้ใช้สามารถเพิ่มพนักงาน ได้โดยการกดปุ่ม Add Employee จากนั้นกรอกข้อมูลให้เรียบร้อย แล้วกดปุ่ม Add เป็นอันเสร็จสิ้น

4.4หน้าต่างเพิ่มเมนูอาหาร (Add Menu)




ภาพที่ 16

คำอธบาย : ผู้ใช้สามารถเพิ่มข้อมูลเกี่ยวเมนูอาหาร ได้โดยการกดปุ่ม Add Menu จากนั้นกรอกข้อมูลให้เรียบร้อย แล้วกดปุ่ม Add เป็นอันเสร็จสิ้น

4.5 หน้าต่างรายการอาหารทั้งหมด (Menu list)




ภาพที่ 17

คำอธบาย : ผู้ใช้สามารถดูและค้นหาเมนูทั้งหมดได้ โดยการกดที่ปุ่ม Menu List

# การใช้แนวคิด Inheritance ในการพัฒนา

เป็นการ inheritance มาจากคลาส User เพื่อนำตัวแปล userId , name , password มาตรวจสอบก่อนที่จะเข้าใช้งานเมนูต่าง ๆ หากไม่มีข้อมูล หรือกรอกข้อมูลไม่ถูกต้อง ก็จะไม่สามารถใช้งานโปรแกรมได้

```
public class User {
    private int userId;
    private String name;
    private String password;
    private String role;

    public User(int userId, String name, String password, String role) {
        this.userId = userId;
        this.name = name;
        this.password = password;
        this.role = role;
    }

    public String getRole() {
        return role;
    }

    public boolean authenticate(String username, String password) {
        return this.name.equals(username) && this.password.equals(password);
    }

    public void setRole(String role) {
        this.role = role;
    }

    public int getUserId() {
        return userId;
    }




    public String getPassword() {
        return password;
    }

    public void setUserId(int userId) {
        this.userId = userId;
    }

    public void setName(String name) {
        this.name = name;
    }

    public void setPassword(String password) {
        this.password = password;
    }
```

```java
public class Employee extends User {
    private JFrame frame;
    private JPanel contentPanel;
    private CardLayout cardLayout;

    public Employee(int userId, String name, String password) {
        super(userId, name, password, "employee");
        initUI();
    }
```

```java
public class Chef extends User {

    private JFrame frame;
    private HashMap<String, JButton> tableButtons = new HashMap<>();
    private CardLayout cardLayout;
    private JPanel mainPanel;

    public Chef(int userId, String name, String password) {
        super(userId, name, password, "Chef");
```

```java
public class Manager extends User {

    private JFrame frame;
    private CardLayout cardLayout;
    private JPanel contentPanel;
    private JTable table;
    private DefaultTableModel model;
    private int selectedId = -1;
    private JTextField searchField;

    public Manager(int userId, String username, String password) {
        super(userId, username, password, "Manager");
```

```java
  public void routeToRole(JLabel warningLabel) {
      String username = usernameField.getText().trim();
      String password = new String(passwordField.getPassword()).trim();

      if (username.isEmpty() || password.isEmpty()) {
          warningLabel.setText("กรุณากรอกชื่อผู้ใช้และรหัสผ่าน");
          return;
      }

      try (Connection conn = DatabaseManager.connect();
           PreparedStatement stmt = conn.prepareStatement(
                   "SELECT id, username, password, role FROM users WHERE username = ? AND password = ?")) {

          stmt.setString(1, username);
          stmt.setString(2, password);
          ResultSet rs = stmt.executeQuery();

          if (rs.next()) {
              int userId = rs.getInt("id");
              String uname = rs.getString("username");
              String pwd = rs.getString("password");
              String role = rs.getString("role");

              dispose();
              switch (role.toLowerCase()) {
                  case "manager": new Manager(userId, uname, pwd); break;
                  case "employee": new Employee(userId, uname, pwd); break;
                  case "chef": new Chef(userId, uname, pwd); break;
                  default: JOptionPane.showMessageDialog(this, "บทบาทผู้ใช้ไม่ถูกต้อง");

          }
      } else {
          warningLabel.setText("ชื่อผู้ใช้หรือรหัสผ่านไม่ถูกต้อง");
      }

  } catch (Exception e) {
      e.printStackTrace(); // แสดงข้อความ error แบบเต็มใน console
      JOptionPane.showMessageDialog(this, "เกิดข้อผิดพลาดในการเชื่อมต่อฐานข้อมูล:\n" + e.getMessage());
  }
```

# การใช้แนวคิด Composition ในการพัฒนา

1. เป็นการ Composition กันของคลาส Order และ OrderItem โดยการจะทำอาหารแต่ละครั้งของเชฟจำเป็นจะต้องมีวัตถุดิบที่เพียงพอต่อการทำอาหาร ในการสั่งอาหาร (Order) ทุกครั้งจึงจำเป็นจะต้องมีวัตถุดิบ (OrdersItem)

```
public class Order {
    private Table table;
    private Timestamp orderTime;
    private String menuCode;
    private String menuName;
    private int price;
    private int quantity;
    private String remark;

    // Constructor สำหรับออเดอร์ที่ไม่มีรายละเอียดเมนู
    public Order(Table table, Timestamp orderTime) {
        this.table = table;
        this.orderTime = orderTime;
    }

    // Constructor สำหรับออเดอร์ที่มีรายละเอียดเมนู + remark
    public Order(Table table, String menuCode, String menuName, int price, int quantity, Timestamp orderTime, String remark) {
        this.table = table;
        this.menuCode = menuCode;
        this.menuName = menuName;
        this.price = price;
        this.quantity = quantity;
        this.orderTime = orderTime;
        this.remark = remark;
    }
```

```
   public Order(Table table, String menuCode, String menuName, int price, int quantity, Timestamp orderTime) {
       this(table, menuCode, menuName, price, quantity, orderTime, "");
   }

   public Table getTable() {
       return table;
   }

   public Timestamp getOrderTime() {
       return orderTime;
   }

   public String getFormattedTime() {
       if (orderTime == null) return "";
       return new SimpleDateFormat("HH:mm:ss").format(orderTime);
   }

   public String getMenuCode() {
       return menuCode;
   }

   public String getMenuName() {
       return menuName;
   }

   public int getPrice() {
       return price;
   }


   public int getQuantity() {
       return quantity;
   }

   public String getRemark() {
       return remark;
   }

   public void setRemark(String remark) {
       this.remark = remark;
   }
```

```java
public class OrderItem {
    private MenuItem menu;
    private int quantity;

    public OrderItem(MenuItem menu, int quantity) {
        this.menu = menu;
        this.quantity = quantity;
    }


    public MenuItem getMenu() {
        return menu;
    }

    public int getQuantity() {
        return quantity;
    }


    public double getSubtotal() {

        if (menu != null) {
            return menu.getPrice() * quantity;
        } else {
            return 0.0;
        }
    }
```

```java
   public void setQuantity(int quantity) {
       this.quantity = quantity;
   }
```

2. เป็นการ Composition กันของคลาส Order และ Queue การที่เชฟจะทราบถึงลำดับคิวของอาหารที่สั่งในแต่ละโต๊ะได้ จำเป็นจะต้องมี รายการอาหารที่สั่ง (Order) เข้ามาก่อน จึงจะรับทราบรันคิว (Queue) การทำอาหารได้

```java
  private void loadOrdersFromDatabase() {
      tableOrdersMap.clear();

      try (Connection conn = DriverManager.getConnection("jdbc:mysql://localhost:3306/restaurant_db", "root", "");
           PreparedStatement stmt = conn.prepareStatement(
                   "SELECT * FROM chef_orders WHERE status = 'ออเดอร์' AND updated_at IN (" +
                           "SELECT MAX(updated_at) FROM chef_orders WHERE status = 'ออเดอร์' GROUP BY table_number" +
                   ") ORDER BY table_number, updated_at DESC")) {

          ResultSet rs = stmt.executeQuery();

          while (rs.next()) {
              String tableNum = rs.getString("table_number");
              String menuCode = rs.getString("menu_code");
              String menuName = rs.getString("menu_name");
              int quantity = rs.getInt("quantity");
              Timestamp time = rs.getTimestamp("updated_at");
              String remark = rs.getString("remark");

              int price = getMenuPrice(conn, menuCode);

              Table table = new Table(tableNum);
              Order order = new Order(table, menuCode, menuName, price, quantity, time, remark);

              tableOrdersMap.putIfAbsent(tableNum, new ArrayList<>());
              tableOrdersMap.get(tableNum).add(order);
          }
```

```
    } catch (SQLException ex) {
        ex.printStackTrace();
        JOptionPane.showMessageDialog(this, "ไม่สามารถโหลดคิวจากฐานข้อมูลได้\n" + ex.getMessage());
    }

    updateOrdersPanel();
}

private int getMenuPrice(Connection conn, String code) {
    try (PreparedStatement stmt = conn.prepareStatement("SELECT price FROM menu WHERE code = ?")) {
        stmt.setString(1, code);
        ResultSet rs = stmt.executeQuery();
        if (rs.next()) {
            return rs.getInt("price");
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
    return 0;
}
```

# การใช้แนวคิด Polymorphism ในการพัฒนา

ระบบถูกออกแบบโดยใช้แนวคิด Polymorphism ซึ่งเมธอดหลักชื่อ execute() ถูกกำหนดในแต่ละคลาส เพื่อให้ทำหน้าที่เฉพาะของแต่ละหน้าจอ เช่น AddEmployee ใช้ execute() เพื่อเพิ่มพนักงาน , MenuCategory ใช้ execute() เพื่อเพิ่มเมนูอาหาร , MenuList ใช้ execute() เพื่อแสดงรายการเมนู , และ DailyReport ใช้ execute() เพื่อโหลดรายงานยอดขายประจำวัน

```
public void execute(String panelName) {
    Component panel = findPanelByName(panelName);
    if (panel == null) {
        System.out.println("ไม่พบ panel: " + panelName);
        return;
    }

    try {
        panel.getClass().getMethod("execute").invoke(panel);
    } catch (NoSuchMethodException e) {
        System.out.println("ไม่มีเมธอด execute() ในคลาส: " + panel.getClass().getSimpleName());
    } catch (Exception e) {
        e.printStackTrace();
    }
}
```

```
public void execute() {
    model.setRowCount(0);
    dataset.clear();

    try (Connection conn = DriverManager.getConnection("jdbc:mysql://localhost:3306/restaurant_db", "root", "");
         PreparedStatement stmt = conn.prepareStatement(
                 "SELECT m.name, m.price, SUM(co.quantity) AS total_qty, " +
                         "GROUP_CONCAT(DISTINCT co.remark SEPARATOR ', ') AS remarks " +
                         "FROM chef_orders co " +
                         "JOIN menu m ON co.menu_code = m.code " +
                         "WHERE DATE(co.updated_at) = CURDATE() " +
                         "GROUP BY m.code, m.name, m.price")) {

        ResultSet rs = stmt.executeQuery();

        double totalSales = 0.0;
        while (rs.next()) {
            String name = rs.getString("name");
            int price = rs.getInt("price");
            int qty = rs.getInt("total_qty");
            String remark = rs.getString("remarks");
            int total = price * qty;
            totalSales += total;

            dataset.setValue(name, total);
            model.addRow(new Object[]{name, price, qty, total, remark != null ? remark : "-"});
        }

        saleTotal.setText("Daily sales : " + String.format("%,.2f", totalSales) + " Bath");

    } catch (Exception e) {
        e.printStackTrace();
        JOptionPane.showMessageDialog(this, "ไม่สามารถโหลดข้อมูลยอดขายได้");
    }
}
```

```java
    // เมธอดหลักของ AddEmployee
    public void execute() {
        try (Connection conn = DriverManager.getConnection("jdbc:mysql://localhost:3306/restaurant_db", "root", "");
             PreparedStatement stmt = conn.prepareStatement(
                     "INSERT INTO users (id, username, firstname, lastname, password, role) VALUES (?, ?, ?, ?, ?, ?)")) {

            stmt.setInt(1, Integer.parseInt(idField.getText().trim()));
            stmt.setString(2, userField.getText().trim());
            stmt.setString(3, fnameField.getText().trim());
            stmt.setString(4, lnameField.getText().trim());
            stmt.setString(5, new String(passField.getPassword()));
            stmt.setString(6, roleField.getText().trim());

            stmt.executeUpdate();
            JOptionPane.showMessageDialog(this, "เพิ่มพนักงานสำเร็จแล้ว");

            idField.setText("");
            userField.setText("");
            fnameField.setText("");
            lnameField.setText("");
            passField.setText("");
            roleField.setText("");

        } catch (Exception ex) {
            ex.printStackTrace();
            JOptionPane.showMessageDialog(this, "เกิดข้อผิดพลาดในการบันทึกข้อมูล");
        }
    }
}
```

```java
    // เมธอดหลักของ MenuCategory
    public void execute() {
        String code = codeField.getText().trim();
        String name = nameField.getText().trim();
        String priceText = priceField.getText().trim();

        if (code.isEmpty() || name.isEmpty() || priceText.isEmpty()) {
            JOptionPane.showMessageDialog(this, "กรุณากรอก Code, ชื่อเมนู และราคา");
            return;
        }

        try {
            int price = Integer.parseInt(priceText);

            try (Connection conn = DriverManager.getConnection("jdbc:mysql://localhost:3306/restaurant_db", "root", "");
                 PreparedStatement stmt = conn.prepareStatement("INSERT INTO menu (code, name, price) VALUES (?, ?, ?)")) {

                stmt.setString(1, code);
                stmt.setString(2, name);
                stmt.setInt(3, price);
                stmt.executeUpdate();

                JOptionPane.showMessageDialog(this, "เพิ่มเมนูเรียบร้อยแล้ว");


                for (Component comp : parentPanel.getComponents()) {
                    if (comp instanceof MenuList) {
                        ((MenuList) comp).execute();
                        break;
                    }
                }

                cardLayout.show(parentPanel, "Menu List");

            } catch (SQLIntegrityConstraintViolationException ex) {
                JOptionPane.showMessageDialog(this, "Code นี้มีอยู่แล้ว กรุณาเปลี่ยนใหม่");
            } catch (Exception ex) {
                ex.printStackTrace();
                JOptionPane.showMessageDialog(this, "เกิดข้อผิดพลาดในการเพิ่มเมนู");
            }

        } catch (NumberFormatException ex) {
            JOptionPane.showMessageDialog(this, "กรุณากรอกราคาเป็นตัวเลข");
        }
    }
```

```java
public void execute() {
    model.setRowCount(0);
    String keyword = searchField.getText().trim();

    try (Connection conn = DriverManager.getConnection("jdbc:mysql://localhost:3306/restaurant_db", "root", "");
         PreparedStatement stmt = conn.prepareStatement("SELECT * FROM menu WHERE name LIKE ? OR code LIKE ?")) {

        String pattern = "%" + keyword + "%";
        stmt.setString(1, pattern);
        stmt.setString(2, pattern);

        ResultSet rs = stmt.executeQuery();
        while (rs.next()) {
            String code = rs.getString("code");
            String name = rs.getString("name");
            int price = rs.getInt("price");

            model.addRow(new Object[]{code + " " + name, price + " บาท", "Edit", "Delete"});
        }

    } catch (Exception e) {
        e.printStackTrace();
    }
}
```

# การใช้แนวคิด Search ในการพัฒนา

1. ระบบในคลาส Employee ถูกออกแบบโดยใช้แนวคิด Search LIKE ผ่าน SQL เพื่อให้ผู้ใช้ค้นหาเมนูได้จากคำบางส่วน และแสดงผล เพิ่มความสะดวกและตอบสนองได้ทันที โดยมีตัวแปร code และ name

```
        ArrayList<JButton> menuButtons = new ArrayList<>();

        ActionListener loadMenu = e -> {
            menuPanel.removeAll();
            menuButtons.clear();
            String keyword = searchField.getText().trim().toLowerCase();

            try (Connection conn = DriverManager.getConnection("jdbc:mysql://localhost:3306/restaurant_db", "root", "");
                 PreparedStatement stmt = conn.prepareStatement(
                         "SELECT code, name FROM menu WHERE code LIKE ? OR name LIKE ?")) {

                String pattern = "%" + keyword + "%";
                stmt.setString(1, pattern);
                stmt.setString(2, pattern);

                ResultSet rs = stmt.executeQuery();

                int x = 0, y = 0;
                while (rs.next()) {
                    String label = rs.getString("code") + " " + rs.getString("name");
                    JButton btn = new JButton(label);
                    btn.setBounds(x, y, 250, 60);
                    btn.setBackground(new Color(240, 230, 255));
                    menuPanel.add(btn);
                    menuButtons.add(btn);

                    x += 290;
                    if (menuButtons.size() % 3 == 0) {
                        x = 0;
                        y += 80;
                    }
                }

            } catch (Exception ex) {
                ex.printStackTrace();
                JOptionPane.showMessageDialog(panel, "โหลดเมนูไม่สำเร็จ");
            }

            menuPanel.repaint();
            menuPanel.revalidate();
        };

        searchBtn.addActionListener(loadMenu);
        loadMenu.actionPerformed(null);

        return panel;
    }
```

2. ระบบในคลาส OrdersMenuSelection ถูกออกแบบโดยใช้แนวคิด Search เพื่อให้สามารถค้นหาเมนูอาหารจากฐานข้อมูลด้วย SQL LIKE และแสดงผลลัพธ์ตามคำค้นของผู้ใช้โดยมีตัวแปรเป็น code , name และ price

```
private void loadMenuFromDatabase(String keyword) {
    menuPanel.removeAll();

    try (Connection conn = DriverManager.getConnection("jdbc:mysql://localhost:3306/restaurant_db", "root", "");
         PreparedStatement stmt = conn.prepareStatement(
                 "SELECT code, name, price FROM menu WHERE code LIKE ? OR name LIKE ?")) {

        String keyPattern = "%" + keyword + "%";
        stmt.setString(1, keyPattern);
        stmt.setString(2, keyPattern);

        ResultSet rs = stmt.executeQuery();
        int x = 0, y = 0;

        while (rs.next()) {
            String code = rs.getString("code");
            String name = rs.getString("name");
            int price = rs.getInt("price");

            JButton btn = new JButton(code + " " + name);
            btn.setBounds(x, y, 250, 60);
            btn.setBackground(new Color(240, 230, 255));
            btn.setFocusPainted(false);

            btn.addActionListener(e -> openQuantityDialog(code, name, price, btn));
            menuPanel.add(btn);

            x += 290;
            if (x > 800) {
                    ^

                y += 80;
              }
          }

      } catch (Exception ex) {
          ex.printStackTrace();
          JOptionPane.showMessageDialog(this, "โหลดเมนูไม่สำเร็จ");
      }

      menuPanel.repaint();
      menuPanel.revalidate();
  }
```

3.ระบบในคลาส Manager ถูกออกแบบโดยใช้แนวคิด Search เพื่อให้สามารถค้นหาเมนูอาหารจากฐานข้อมูลด้วย SQL LIKE และแสดงผลลัพธ์ตามคำค้นของผู้ใช้โดยมีเมธอด getInt กับ getString

```
private void loadEmployees(String keyword) {
    model.setRowCount(0);
    String sql = "SELECT * FROM users";
    boolean hasKeyword = keyword != null && !keyword.trim().isEmpty();

    if (hasKeyword) {
        sql += " WHERE username LIKE ? OR firstname LIKE ? OR lastname LIKE ?";
    }

    try (Connection conn = DriverManager.getConnection("jdbc:mysql://localhost:3306/restaurant_db", "root", "");
         PreparedStatement stmt = conn.prepareStatement(sql)) {

        if (hasKeyword) {
            String pattern = "%" + keyword + "%";
            stmt.setString(1, pattern);
            stmt.setString(2, pattern);
            stmt.setString(3, pattern);
        }

        ResultSet rs = stmt.executeQuery();
        boolean found = false;

        while (rs.next()) {
            found = true;
            model.addRow(new Object[]{
                rs.getInt("id"),
                rs.getString("username"),
                rs.getString("firstname"),
                rs.getString("lastname"),
                rs.getString("password"),
                rs.getString("role")
            });
        }

        if (!found) {
            JOptionPane.showMessageDialog(frame, "ไม่พบข้อมูลพนักงาน", "แจ้งเตือน", JOptionPane.INFORMATION_MESSAGE);
        }

    } catch (Exception e) {
        e.printStackTrace();
        JOptionPane.showMessageDialog(frame, "เกิดข้อผิดพลาดในการโหลดข้อมูลพนักงาน");
    }
}
```

4.ระบบในคลาส MenuList ถูกออกแบบโดยใช้แนวคิด Search เพื่อให้สามารถค้นหาเมนูอาหารจากฐานข้อมูลด้วย SQL LIKE และแสดงผลลัพธ์ตามคำค้นของผู้ใช้โดยมีตัวแปรเป็น code , name และ price

```
public void execute() {
    model.setRowCount(0);
    String keyword = searchField.getText().trim();

    try (Connection conn = DriverManager.getConnection("jdbc:mysql://localhost:3306/restaurant_db", "root", "");
         PreparedStatement stmt = conn.prepareStatement("SELECT * FROM menu WHERE name LIKE ? OR code LIKE ?")) {

        String pattern = "%" + keyword + "%";
        stmt.setString(1, pattern);
        stmt.setString(2, pattern);

        ResultSet rs = stmt.executeQuery();
        while (rs.next()) {
            String code = rs.getString("code");
            String name = rs.getString("name");
            int price = rs.getInt("price");

            model.addRow(new Object[]{code + " " + name, price + " บาท", "Edit", "Delete"});
        }

    } catch (Exception e) {
        e.printStackTrace();
    }
}
```